

Devoir en pratique de l'optimisation numérique – M2 ISC 903.

Résolution d'un problème inverse: déconvolution d'un signal triangulaire

Objectif de l'exercice

Le problème considéré est celui de la reconstruction d'un signal $x(t)$ (monodimensionnel), supposé inconnu car non accessible à une mesure directe, ayant subi un filtrage gaussien (convolution) inhérent à l'instrument de mesure. Il s'agit d'un filtrage linéaire d'usage courant en traitement d'image. La réponse impulsionnelle d'un filtre gaussien est définie par :

$$h(t) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(t-m)^2}{2\sigma^2}\right) \quad (1)$$

- Où m et σ^2 sont la moyenne et la variance de la gaussienne, respectivement.

Le signal y mesuré en sortie du filtre, est supposé être corrompu par des incertitudes additives. En effet, les mesures en sortie de filtre sont à minima sujettes au bruit de quantification dû au convertisseur analogique-numérique de la chaîne de mesure. Il s'agit d'appréhender le caractère instable de l'opération de déconvolution, lorsque celle-ci est effectuée de façon « naïve », soit par division polynomiale (fonction `deconv` de Matlab), soit par filtrage inverse (dans le domaine fréquentiel ou par moindres carrés qui est l'équivalent matriciel).

En présence d'erreurs (de bruit) sur les données, même très faibles, les résultats fournis par ces méthodes d'inversion peuvent, en effet, être inexploitable. Il s'agit là d'un problème inverse mal posé. La réponse en fréquence du système à inverser permet de caractériser le degré de difficulté du problème. L'exercice illustrera la (les) solution(s) obtenues par régularisation linéaire quadratique.

Table of Contents

Devoir en pratique de l'optimisation numérique – M2 ISC 903.....	1
.....	1
Résolution d'un problème inverse: déconvolution d'un signal triangulaire.....	1
Objectif de l'exercice	1
1. Construction du jeu de données	2

a. Simulation du signal d'entrée	2
1. Simuler le signal à temps discret $x[n]$ correspondant, pour une durée d'observation $L = 100s$	2
b. Simulation du filtre	3
2. Pour les besoins de la simulation, déterminer la réponse impulsionnelle $h[n]$ du filtre gaussien obtenue par discrétisation de l'équation (1).	4
c. Simulation du problème direct: première approche	5
5. Simuler le signal de sortie non bruité $y_{nb}[n]$ à l'aide de la fonction conv.....	5
6. Simuler le signal quantifié y et représenter celui-ci à l'aide de la fonction stairs.	6
7. Déterminer et représenter le bruit de quantification $b[n]$	7
d. Simulation du problème direct par approche matricielle	8
8. Déterminer la dimension N_y du y_{nb}	8
9. Construire la matrice de convolution H à l'aide de la fonction toeplitz.	8
10. Déterminer la sortie non bruitée.....	11
2. Déconvolution.....	12
a. Déconvolution « naïve » et constat de l'instabilité	13
11. Reconstruire le signal d'entrée avec deconv.....	13
12. Reconstruire le signal d'entrée avec y (bruité).....	13
b. Déconvolution par moindres carrés (ℓ_2)	16
14. Calculer le signal déconvolué à l'aide de l'estimateur des moindres carrés.	16
15. Tracer le signal déconvolué et calculer l'énergie de l'erreur de reconstruction.	17
c. Déconvolution par régularisation linéaire quadratique ($\ell_2\ell_2$)	21
Zone des fonctions fait et utilisé dans le TP.....	24
FIN ET EXPORTATION.....	24

1. Construction du jeu de données

La fréquence d'échantillonnage est fixée à la valeur $f_e = 1\text{Hz}$. Les échantillons du signal d'entrée sont notés $x[n]$, ceux de la réponse impulsionnelle du filtre $h[n]$ et ceux du signal mesuré en sortie $y[n]$. Le signal mesuré en sortie est sujet à des incertitudes additives notées $b[n]$ et le problème direct peut alors être modélisé comme suit :

$$y[n] = (x \star h)[n] + b[n] \quad (2)$$

a. Simulation du signal d'entrée

Le signal d'entrée, supposé être triangulaire, est défini par:

$$x(t) = \begin{cases} \frac{t}{L/2}, & \forall t \in \left[0, \frac{L}{2}\right] \\ \frac{-t + L}{L/2}, & \forall t \in \left[\frac{L}{2}, L\right] \end{cases} \quad (3)$$

1. Simuler le signal à temps discret $x[n]$ correspondant, pour une durée d'observation $L = 100s$.

Aide: "remplacer t par nT_e dans l'équation définissant $x(t)$. Il est possible d'utiliser une instruction du type $x=[x_1, x_2]$, où x_1 et x_2 représentent deux tronçons successifs du signal. De manière générale, il est conseillé d'adapter le script ci-dessous qui permet de maîtriser les différents paramètres de la discrétisation "

```
L = 100; % dure d' observation du signal d' entree ( secondes )
fe = 1; % frequence d' echantillonnage (Hz)
Te = 1/ fe ; % periode d' echantillonnage ( secondes )
Nx = L/ Te ; % nombre d' echantillons du signal x
nx = 0: Nx -1; % indice temporel ( entier )
tx = nx * Te ; % base de temps ( secondes )
```

Solution :

```
% Paramètres de la discrétisation
L = 100; % dure d' observation du signal d' entree ( secondes )
fe = 1; % frequence d' echantillonnage (Hz)
Te = 1/ fe ; % periode d' echantillonnage ( secondes )
Nx = L/ Te ; % nombre d' echantillons du signal x
nx = 0: Nx -1; % indice temporel ( entier )
tx = nx * Te ; % base de temps ( secondes )
```

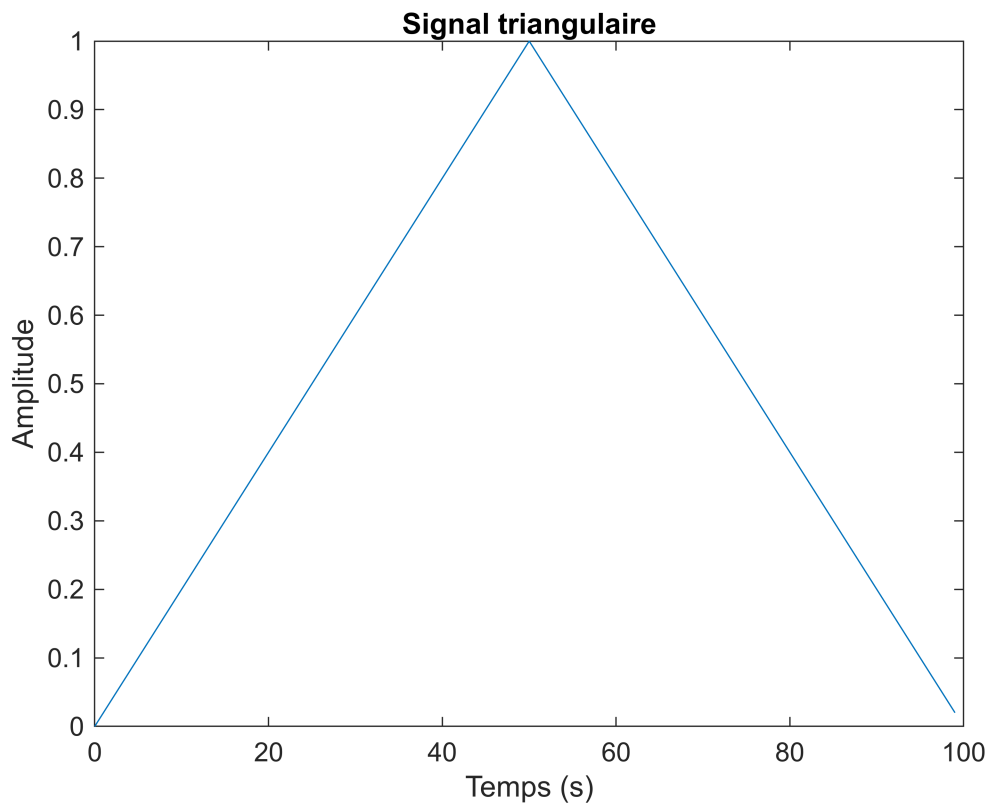
```
x = SignTrigAS(L,tx);
```

```
% Affichage du signal
```

```
disp(x); %tableau de x
```

0 0.0200 0.0400 0.0600 0.0800 0.1000 0.1200 0.1400 0.1600 0.1800 0.2000 0

```
figure;
plot(tx, x); xlabel('Temps (s)');ylabel('Amplitude');title('Signal triangulaire');
```



b. Simulation du filtre

2. Pour les besoins de la simulation, déterminer la réponse impulsionnelle $h[n]$ du filtre gaussien obtenue par discrétisation de l'équation (1).

Le filtre est de plus supposé causal (i.e. , de valeurs nulles aux temps négatifs) et ses paramètres seront pris égaux respectivement à $m = 15$ et $\sigma = 4$. Enfin, la réponse impulsionnelle est tronquée à un horizon de $N_h = 30$ s. Stocker cette réponse impulsionnelle dans un vecteur et la représenter.

Solution :

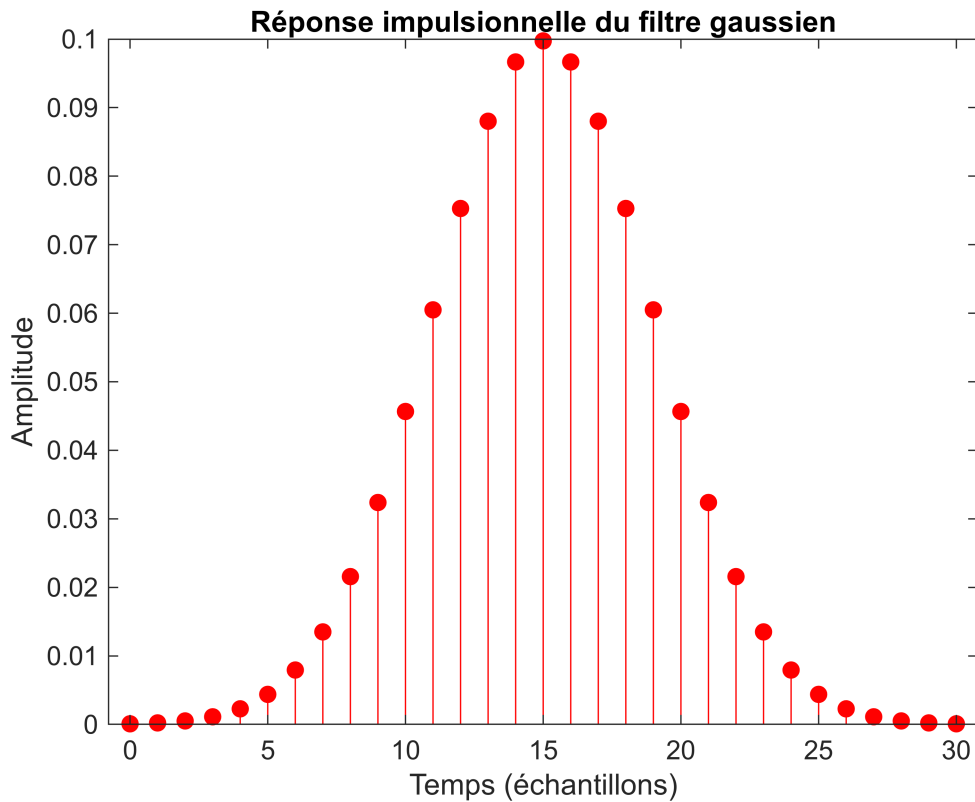
```
% Paramètres du filtre gaussien
m = 15;          % Moyenne
sigma = 4;       % Ecart-type
varh = sigma^2;  % variance de la gaussienne
Nh = 30;        % Horizon de troncature

% Définition de la base de temps pour le filtre (de 0 à Nh)
nh = 0:Nh;
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((nh - m).^2) / (2 * varh)); %stocker dans h

% Affichage de la réponse impulsionnelle
disp(h); %vecter h
```

0.0001 0.0002 0.0005 0.0011 0.0023 0.0044 0.0079 0.0135 0.0216 0.0324 0.0457 0

```
figure;
stem(nh, h, 'r', 'filled');
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Réponse impulsionnelle du filtre gaussien');
```



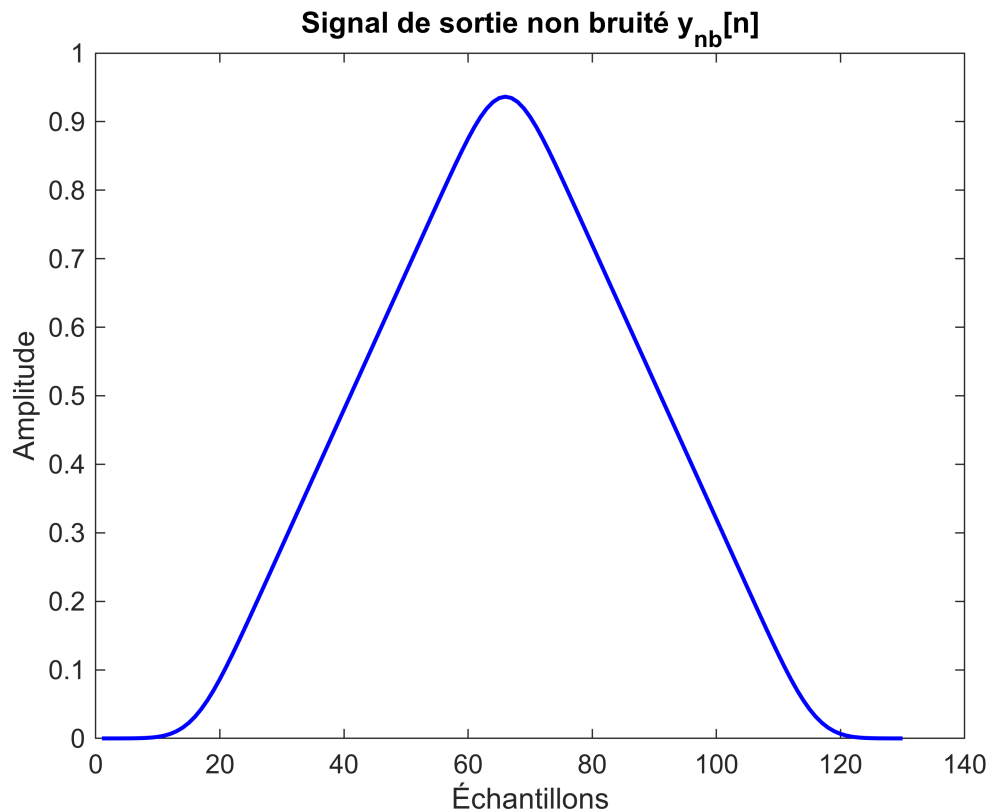
c. Simulation du problème direct: première approche

5. Simuler le signal de sortie non bruité $y_{nb}[n]$ à l'aide de la fonction `conv`.

Comme le filtre gaussien est un filtre linéaire invariant, il est possible de calculer sa sortie comme le produit de convolution discret donné par l'équation (2).

```
% Convolution pour obtenir le signal de sortie non bruité
ynb = conv(x, h);

% Affichage du signal de sortie non bruité
figure;
plot(ynb, 'b', 'LineWidth', 1.5);
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal de sortie non bruité  $y_{nb}[n]$ ');
```



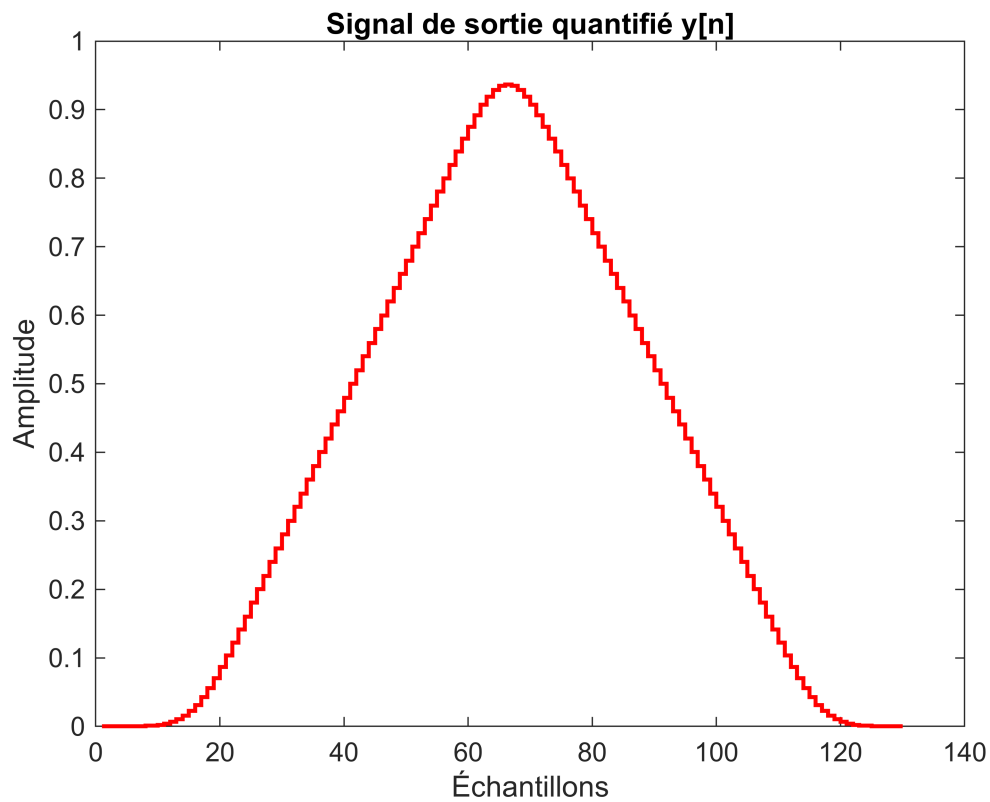
6. Simuler le signal quantifié y et représenter celui-ci à l'aide de la fonction stairs.

Si Le signal de sortie y est mesuré à l'aide d'un convertisseur analogique-numérique, il est sujet au bruit de

quantification. L'opération de quantification peut être simulée par l'instruction $y = \frac{\text{round}(y_{nb} \times 2^{10})}{2^{10}}$.

```
% 1. Ajout du bruit de quantification
y = round(y_nb * 2^10) / 2^10; % Quantification à 10 bits

% 2. Affichage du signal quantifié
figure;
stairs(y, 'r', 'LineWidth', 1.5);
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal de sortie quantifié y[n]');
```



7. Déterminer et représenter le bruit de quantification $b[n]$.

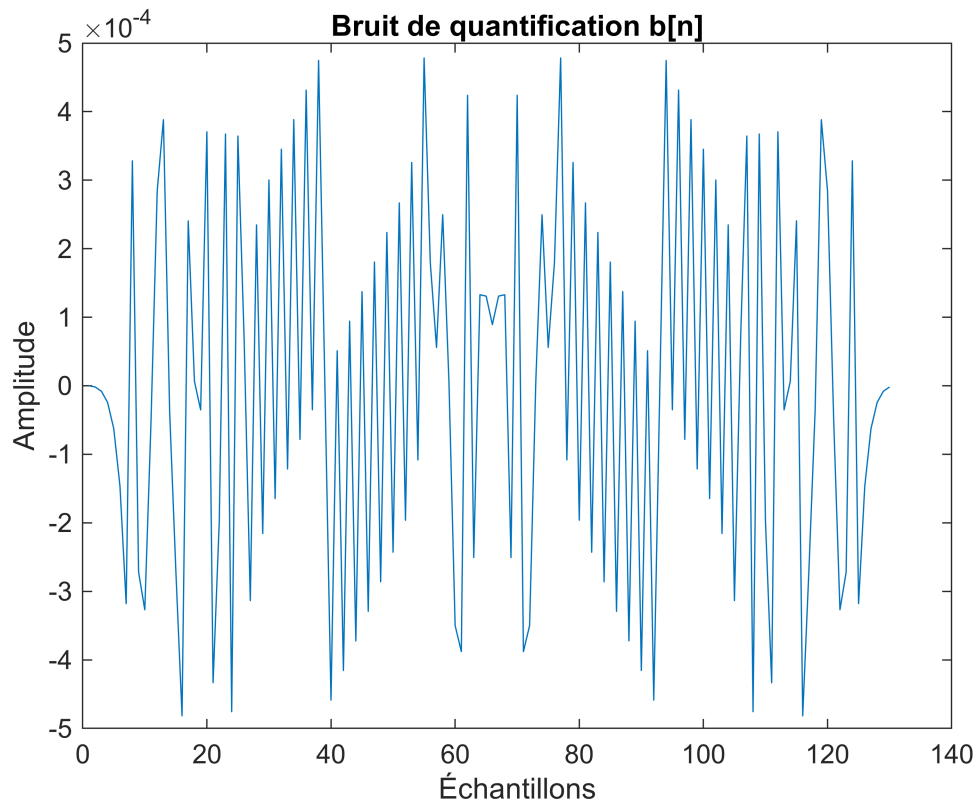
De l'équation 2 : $y[n] = (x \star h)[n] + b[n]$ avec $(x \star h)[n] = y_{nb}$

$y = y_{nb} + b$ d'où :

$b = y - y_{nb}$

```
% 1. Calcul du bruit de quantification
b = y - ynb;

% 2. Affichage du bruit de quantification
figure;
plot(b);
xlabel('Échantillons');
ylabel('Amplitude');
title('Bruit de quantification b[n]');
```



d. Simulation du problème direct par approche matricielle

Dans cette partie, le signal de sortie non bruité est simulé par l'équation matricielle $y_{nb} = Hx$. L'usage d'un paramétrage soigné est recommandé pour permettre de modifier aisément les conditions de simulation et d'affichage des résultats.

8. Déterminer la dimension N_y du y_{nb}

Déterminer la dimension N_y du vecteur de sortie y_{nb} à partir des dimensions N_x et N_h des vecteurs d'entrée x et de réponse impulsionnelle h afin d'obtenir le même nombre d'échantillons que par la fonction `conv`.

```
% Détermination des dimensions
Ny = Nx + Nh -1; % Taille du signal de sortie

% Affichage des tailles
fprintf('Dimension du signal de sortie y_{nb} : %d échantillons\n', Ny);
```

Dimension du signal de sortie y_{nb} : 129 échantillons

9. Construire la matrice de convolution H à l'aide de la fonction `toeplitz`.

Pour cela, il est proposé d'initialiser le premier vecteur ligne et le premier vecteur colonne de H avec des vecteurs nuls de dimension appropriée à l'aide la fonction `zeros`, puis de remplacer certains de leurs éléments avec les éléments de h .

```
H = zeros(Ny, Nx); % Initialisation de la matrice H avec des zéros
```


10

```

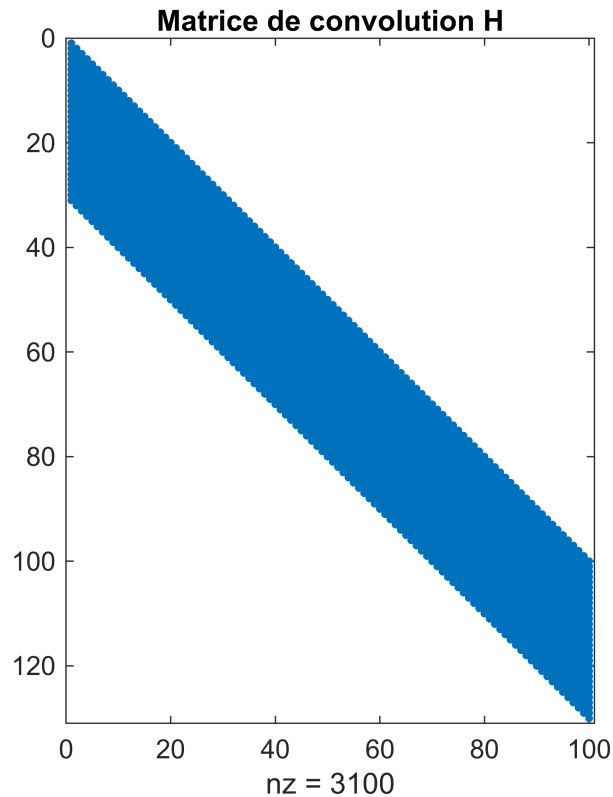
0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0      0      0

```

```

% Affichage de la matrice H
figure;
spy(H); % Affichage de la matrice H sous forme de graphique 'spy'
title('Matrice de convolution H');

```



10. Déterminer la sortie non bruitée

Déterminer la sortie non bruitée par une opération matricielle, puis appliquer la fonction permettant de simuler l'erreur de troncature.

```

% 1. Calcul de la sortie non bruitée avec l'approche matricielle
ynb_matrixH = H*x'; % Multiplication matricielle

erreur_troncature = ynb - ynb_matrixH'; % Erreur entre les deux approches
disp(erreur_troncature);

```

1.0e-15 *

```

0      0      0      0      0      0.0000      0.0001      -0.0001      0.0002      0.0004      -0.0009

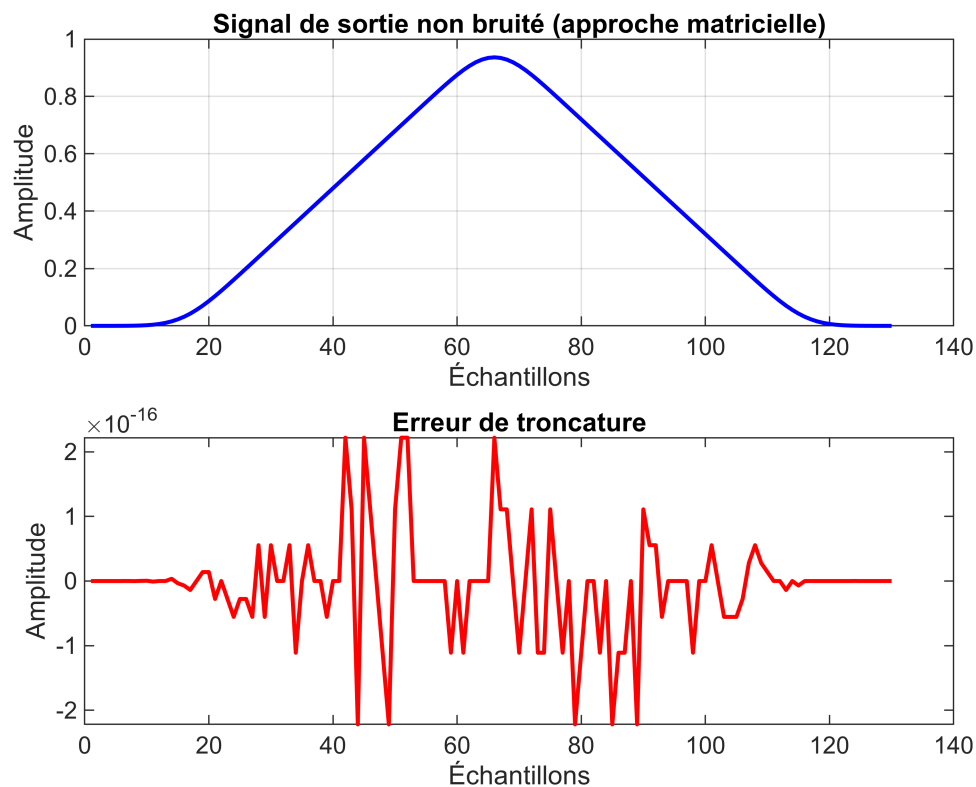
```

```
fprintf('l'erreur moyen de troncature = %d ', mean(erreur_troncature));
```

```
l'erreur moyen de troncature = -4.911749e-18
```

% 3. Affichage des résultats

```
subplot(2,1,1);  
plot(y nb_matrixH, 'b', 'LineWidth', 1.5);  
xlabel('Échantillons');  
ylabel('Amplitude');  
title('Signal de sortie non bruité (approche matricielle)');  
grid on;  
  
subplot(2,1,2);  
plot(erreur_troncature, 'r', 'LineWidth', 1.5);  
xlabel('Échantillons');  
ylabel('Amplitude');  
title('Erreur de troncature');
```



2. Déconvolution

a. Déconvolution « naïve » et constat de l'instabilité

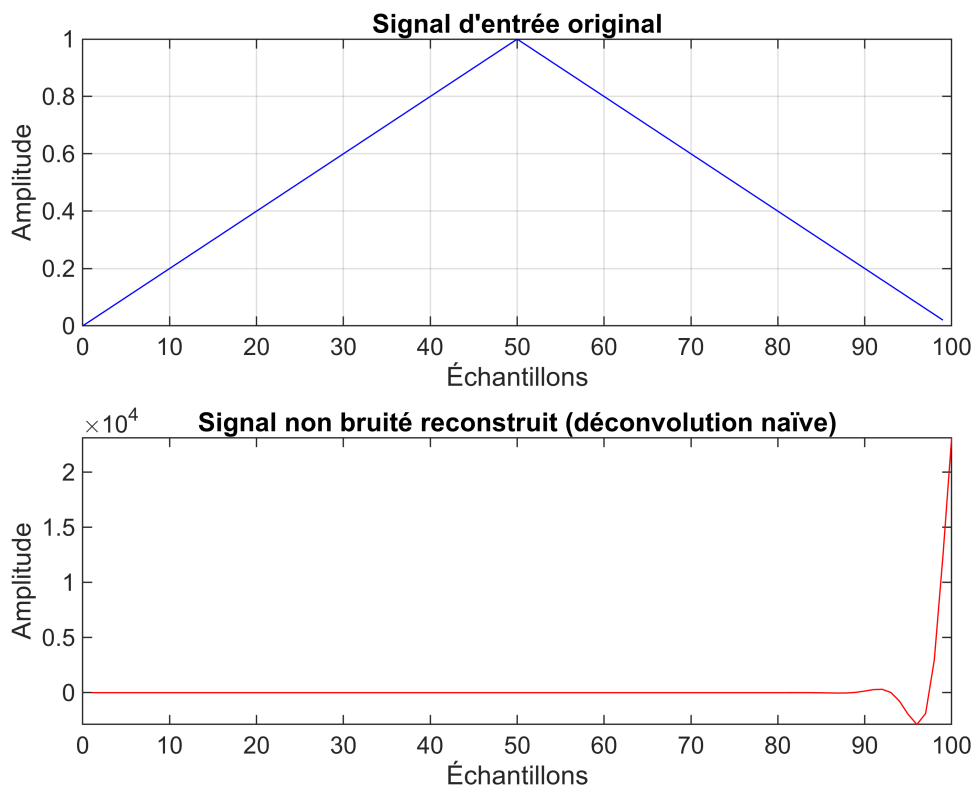
11. Reconstruire le signal d'entrée avec deconv

Reconstruire le signal d'entrée en utilisant l'opération de déconvolution de Matlab (deconv), en se plaçant, dans un premier temps, dans le cas idéal où l'on a accès à la sortie non bruitée ynb.

```
% Déconvolution naïve avec le signal non bruité
[xrecnb, ~] = deconv(ynb, h);

% Affichage du signal reconstruit
figure;
subplot(2,1,1);
plot(tx,x, 'b');
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal d''entrée original');
grid on;

subplot(2,1,2);
plot(xrecnb, 'r');
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal non bruité reconstruit (déconvolution naïve)');
```



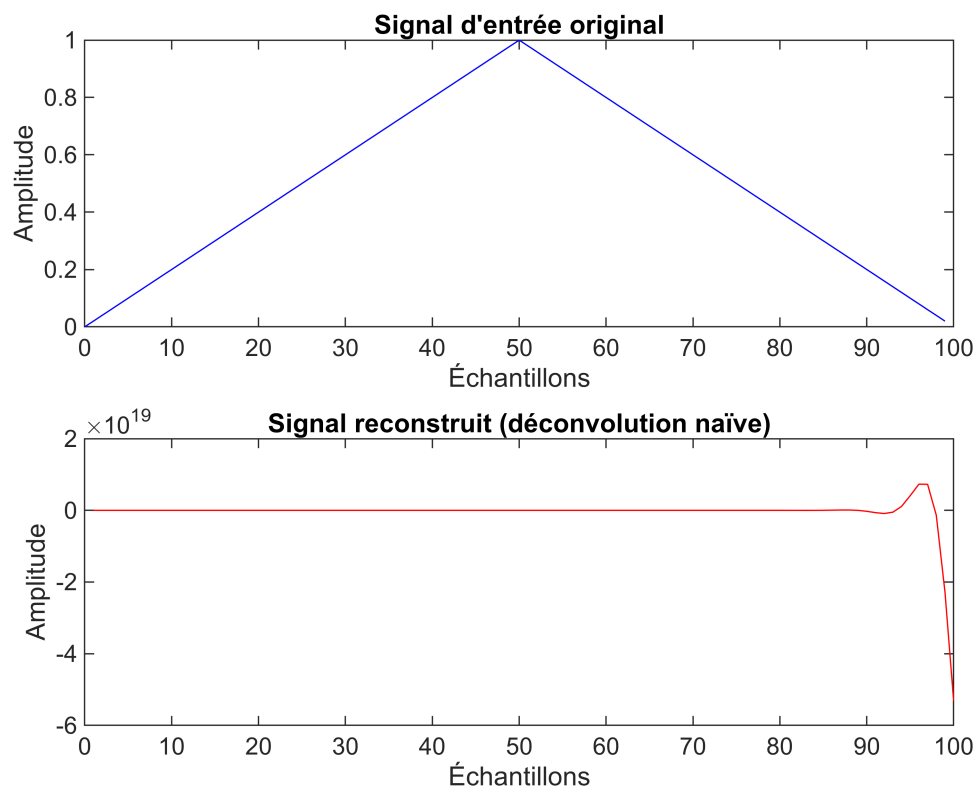
12. Reconstruire le signal d'entrée avec y(bruité)

A présent, reconstruire le signal d'entrée à partir des échantillons mesurés du signal de sortie y (bruité). Le signal reconstruit sera appelé x_{rec}

```
% Déconvolution naïve avec le signal bruité
xrec= deconv(y, h);

% Affichage du signal reconstruitfigure;
figure;
subplot(2,1,1);
plot(tx,x, 'b');
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal d\'entrée original');

subplot(2,1,2);
plot(xrec, 'r');
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal reconstruit (déconvolution naïve)');
```



13.

Tester et commenter l'influence de la « forme » de la réponse impulsionnelle sur la qualité de la reconstruction en modifiant la valeur du paramètre σ .

```

% Tester plusieurs valeurs de sigma
sigma_values = [2, 4, 8, 12]; % Différentes valeurs de sigma
colors = ['b', 'g', 'r', 'm']; % Couleurs pour l'affichage

figure;
hold on;
title('Effet de la valeur de \sigma sur la reconstruction');
for i = 1:length(sigma_values)
    sigma_test = sigma_values(i);

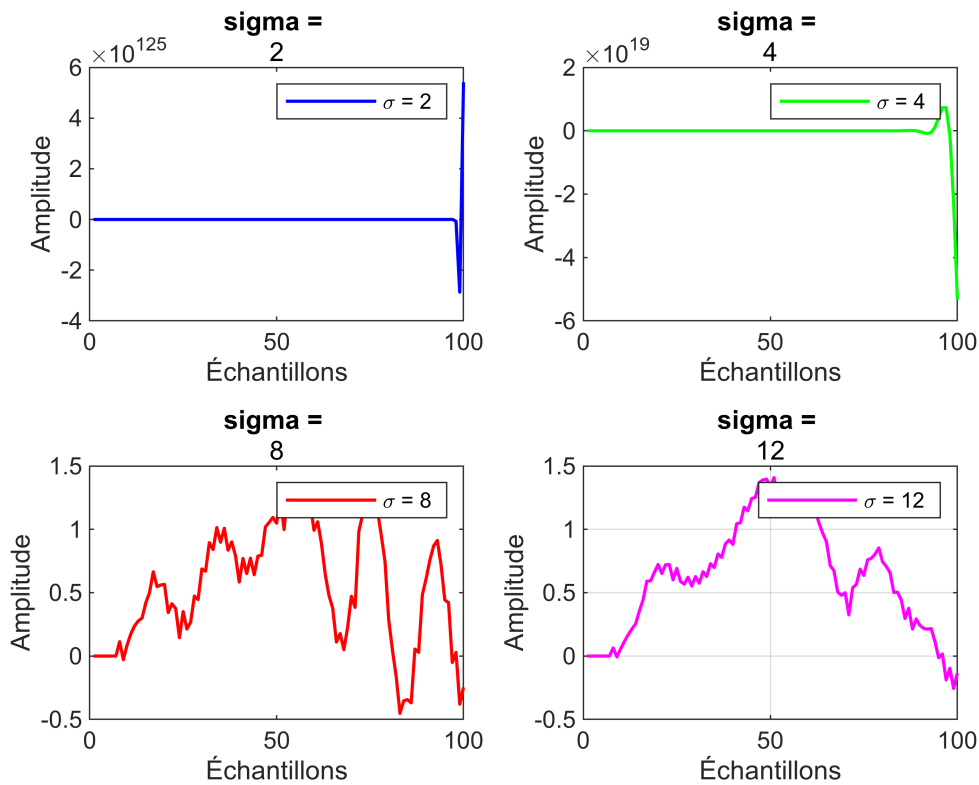
    % Génération d'un nouveau filtre gaussien avec sigma modifié
    h_test = (1 / (sigma_test * sqrt(2 * pi))) * exp(-((nh - m).^2) / (2 *
sigma_test^2));

    % Déconvolution naïve avec le signal bruité y[n]
    [x_rec_test, ~] = deconv(y, h_test);

    % Tracé du signal reconstruit
    subplot(2,2,i);
    plot(x_rec_test, colors(i), 'LineWidth', 1.2, 'DisplayName', ['\sigma = '
num2str(sigma_test)]);
    xlabel('Échantillons');
    ylabel('Amplitude');
    title('sigma = ',sigma_values(i));
    legend show;
end

grid on;
hold off;

```



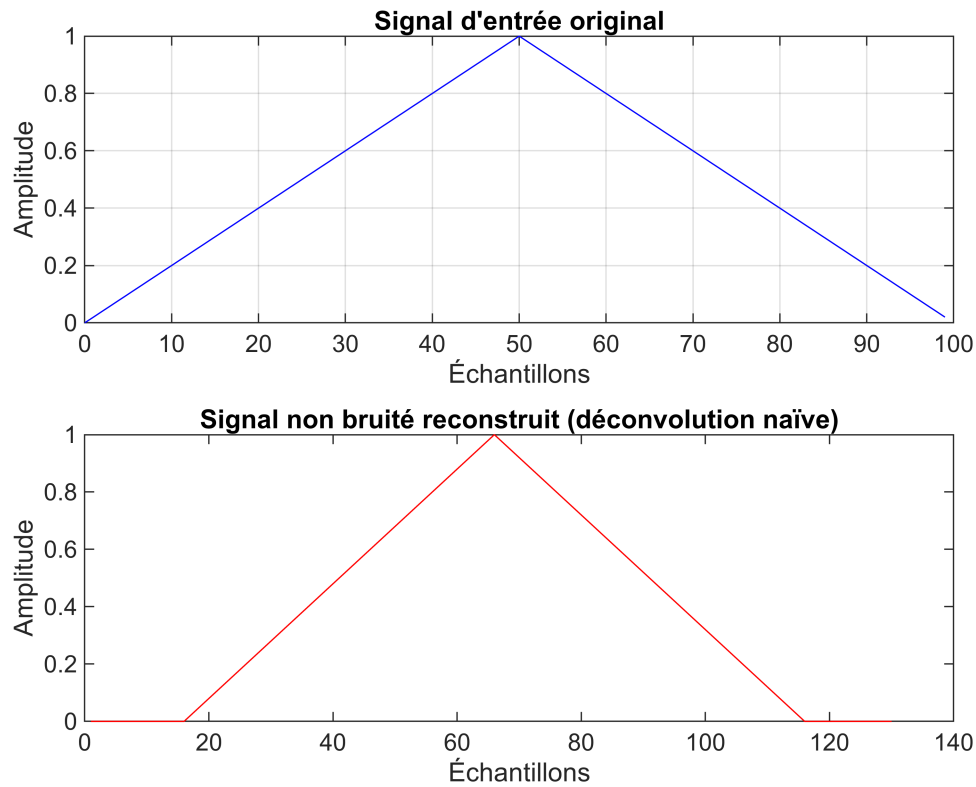
b. Déconvolution par moindres carrés (ℓ_2)

14. Calculer le signal déconvolué à l'aide de l'estimateur des moindres carrés.

```
[xrecl2, ~] = deconv(ynb, h, "same", method="least-squares");

% Affichage du signal reconstruit
figure;
subplot(2,1,1);
plot(tx,x, 'b');
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal d\'entrée original');
grid on;

subplot(2,1,2);
plot(xrecl2, 'r');
xlabel('Échantillons');
ylabel('Amplitude');
title('Signal non bruité reconstruit (déconvolution naïve)');
```

15. Tracer le signal déconvolué et calculer l'énergie de l'erreur de reconstruction.

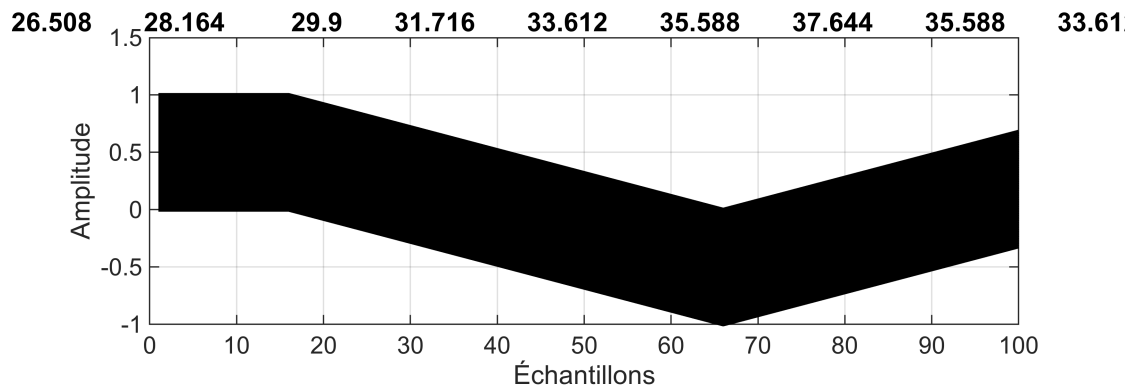
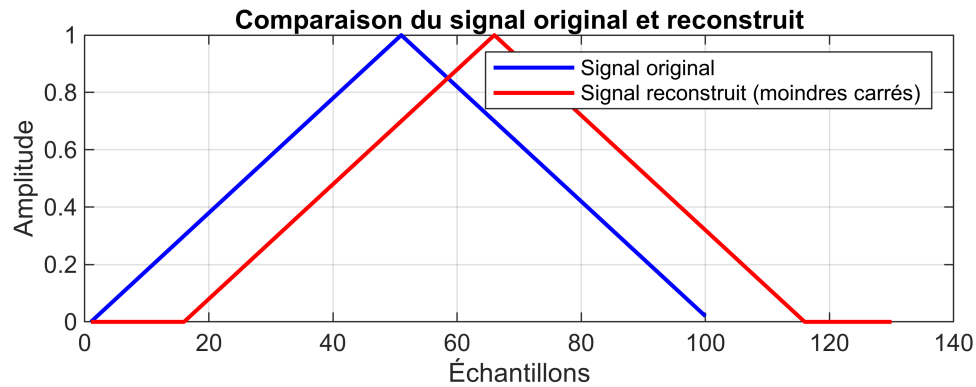
```
% 1. Calcul de l'erreur de reconstruction
erreur_reconstruction = x - xrecl2(1:length(x)); % Ajustement de taille si
nécessaire
energie_erreur = sum(erreur_reconstruction .^ 2); % Énergie de l'erreur

% 2. Affichage du signal reconstruit et de l'erreur
figure;

subplot(2,1,1);
plot(x, 'b', 'LineWidth', 1.5);
hold on;
plot(xrecl2, 'r', 'LineWidth', 1.5);
xlabel('Échantillons');
ylabel('Amplitude');
title('Comparaison du signal original et reconstruit');
legend('Signal original', 'Signal reconstruit (moindres carrés)');
grid on;
hold off;

subplot(2,1,2);
plot(erreur_reconstruction, 'k', 'LineWidth', 1.5);
xlabel('Échantillons');
ylabel('Amplitude');
```

```
title(['Erreur de reconstruction (E = ' num2str(energie_erreur) ')']);
grid on;
```



16.

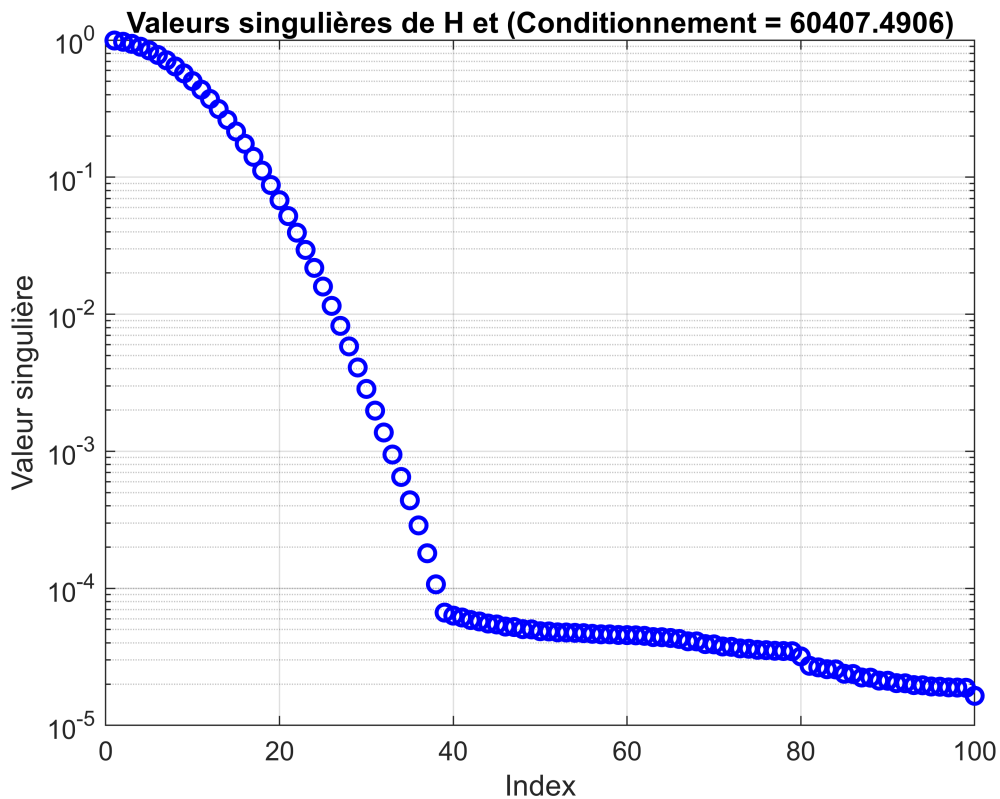
Tracer les valeurs singulières de la matrice H . Que vaut le ratio entre la plus grande valeur singulière, et la plus petite valeur singulière non nulle? Ce quotient est le «conditionnement» de H , et donne une indication quantitative sur la stabilité de la solution des moindres carrés.

```
% 1. Calcul des valeurs singulières
[~,S,~] = svd(H); % Décomposition en valeurs singulières

% 2. Conditionnement de H
sigma_max = max(diag(S));
sigma_min = min(diag(S)); % On ignore les valeurs trop petites (éventuelles erreurs numériques)
conditionnement_H = sigma_max / sigma_min;

% 3. Affichage des valeurs singulières
figure;
semilogy(S, 'bo-', 'LineWidth', 1.5);
xlabel('Index');
ylabel('Valeur singulière');
```

```
title(['Valeurs singulières de H et (Conditionnement = ' num2str(conditionnement_H)
    ')']);
grid on;
```



17. Comment le conditionnement évolue-t-il en fonction de la valeur du paramètre σ du filtre?

```
% Définition des valeurs de sigma à tester
sigma_values = [2, 4, 8, 12, 16, 20, 32];
conditionnement_values = zeros(size(sigma_values));

% Boucle pour tester plusieurs valeurs de sigma
for i = 1:length(sigma_values)
    sigma_test = sigma_values(i);

    % Création d'un nouveau filtre gaussien
    htest = (1 / (sigma_test * sqrt(2 * pi))) * exp(-((nh - m).^2) / (2 *
sigma_test^2));

    % Construction de la matrice de convolution H_test
    Htest = zeros(Ny, Nx); % Initialisation de la matrice H avec des zéros
```

```

% Construction des vecteurs pour toeplitz
ct = [htest'; zeros(Ny - Nh, 1)]; % Première colonne de H
rt = [htest(1), zeros(1, Nx - 1)]; % Première ligne de H

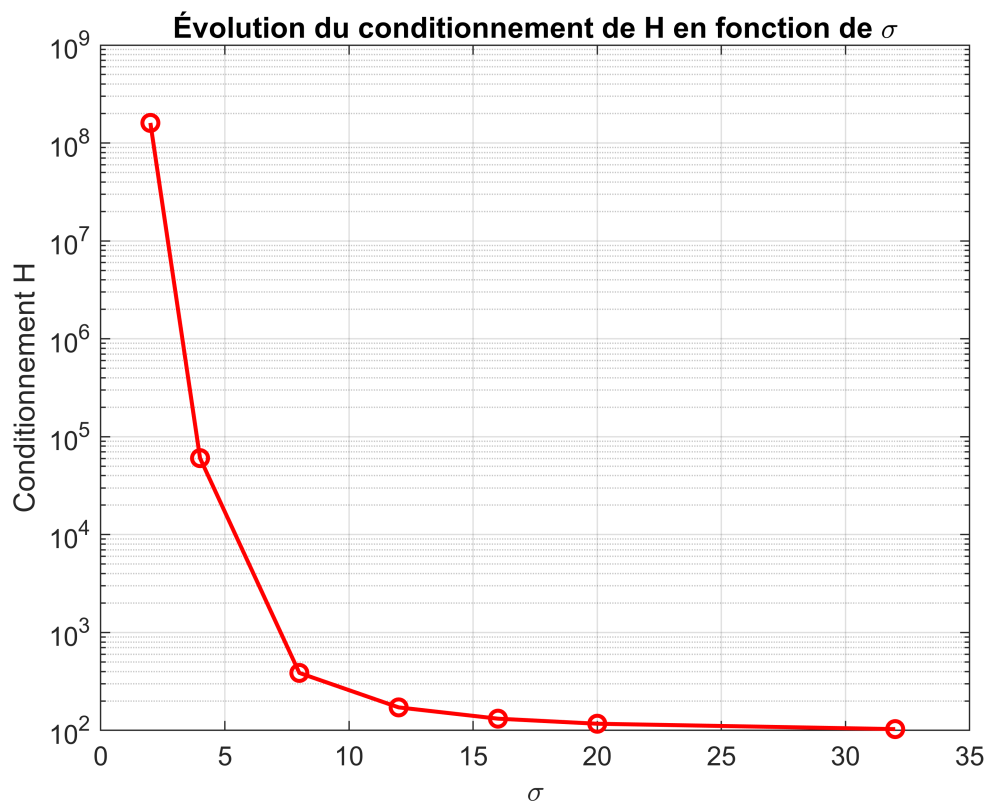
Htest = toeplitz(ct,rt);

% 1. Calcul des valeurs singulières
[~,Stest,~] = svd(Htest); % Décomposition en valeurs singulières

% 2. Conditionnement de H
sigma_maxt = max(diag(Stest));
sigma_mint = min(diag(Stest)); % On ignore les valeurs trop petites
(éventuelles erreurs numériques)
conditionnementHt = sigma_maxt / sigma_mint;
conditionnement_values(i) = conditionnementHt;
end

% Affichage de l'évolution du conditionnement
figure;
semilogy(sigma_values, conditionnement_values, 'ro-', 'LineWidth', 1.5);
xlabel('\sigma');
ylabel('Conditionnement H');
title('Évolution du conditionnement de H en fonction de \sigma');
grid on;

```



c. Déconvolution par régularisation linéaire quadratique (ℓ2ℓ2)

Cette partie concerne la mise en oeuvre de la solution obtenue par minimisation d'un critère d'optimisation composite $J(x) = F_b(x) + \lambda F_r(x)$ (problème régularisé)

18.

Construire la matrice carrée D_1 , de dimension N_x , telle que le produit $D_1 x$ donne les différences premières : $x[n+1] - x[n]$ (les différences premières, lorsqu'elles sont divisées par le pas T_e , correspondent à une approximation de la dérivée première).

Aide :

```
dcol1 = zeros (1 , Nx ) ;  
dcol1 (1:2) = dcol1 (1:2) + [1 -1];  
dlig1 = zeros (1 , Nx ) ;  
dlig1 (1 ,1) = dcol1 (1 ,1) ;  
D1 = toeplitz ( dcol1 , dlig1 );
```

Pour régulariser les moindres carrés, on propose de minimiser la fonctionnelle suivante :

$$J(x) = \frac{1}{2} \|y - Hx\|_2^2 + \lambda \|D_1 x\|_2^2$$

```
dcol1 = zeros (1 , Nx ) ;  
dcol1 (1:2) = dcol1 (1:2) + [1 -1];  
dlig1 = zeros (1 , Nx ) ;  
dlig1 (1 ,1) = dcol1 (1 ,1) ;  
D1 = toeplitz ( dcol1 , dlig1 );
```

19.

Calculer la solution régularisée en déterminant le coefficient de régularisation λ de manière empirique (par exemple en testant plusieurs valeurs dans un intervalle conséquent, comme .

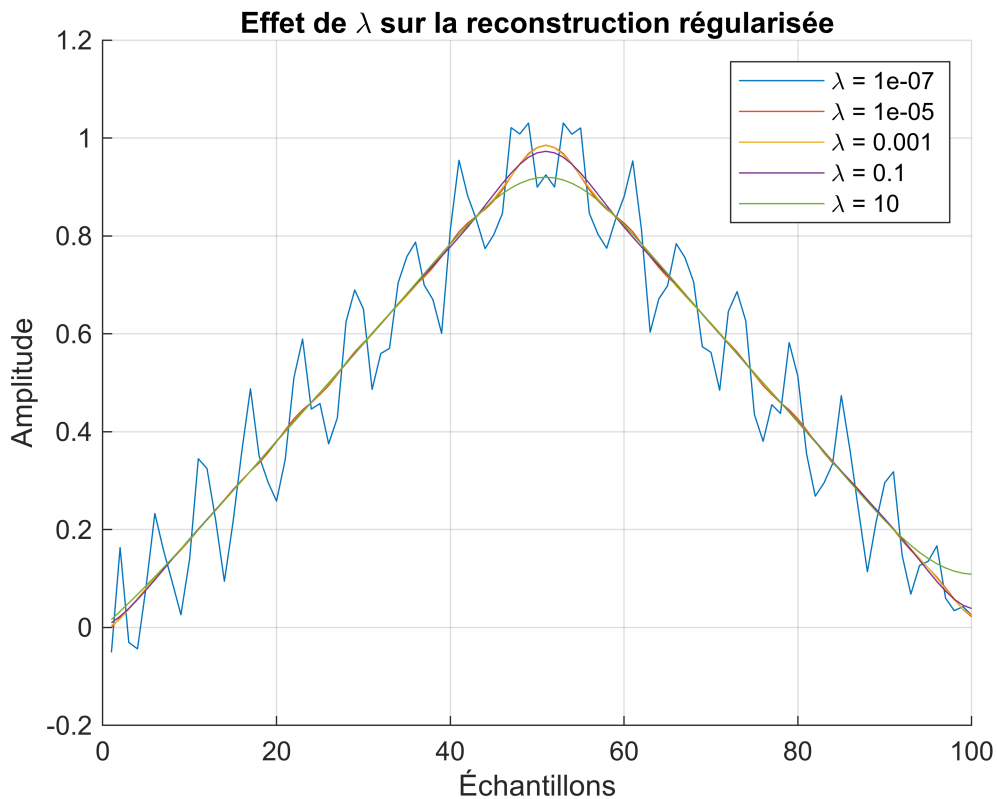
```
% Définition des valeurs de lambda à tester  
lambda_values = logspace(-7, 1, 5); % 10 valeurs entre 10^-7 et 10^2  
  
figure;  
hold on;  
  
for i = 1:length(lambda_values)  
    lambda = lambda_values(i);  
  
    % Résolution du problème régularisé  
    x_rec_reg = (H' * H + lambda * (D1' * D1)) \ (H' * y');
```

```

% Tracé des solutions régularisées
plot(x_rec_reg, 'DisplayName', ['\lambda = ' num2str(lambda)]);
end

xlabel('Échantillons');
ylabel('Amplitude');
title('Effet de \lambda sur la reconstruction régularisée');
legend show;
grid on;
hold off;

```



20.

Le signal à reconstruire étant ici parfaitement connu, il est possible de déterminer la valeur du coefficient de régularisation λ qui minimise l'énergie de l'erreur de reconstruction. Tracer l'évolution de l'énergie de l'erreur de reconstruction en fonction de λ . Ecrire une procédure permettant d'approcher la valeur optimale de λ , par recherche exhaustive sur un intervalle, à l'aide de la fonction `min`.

```

% Définition de l'intervalle de recherche pour lambda
lambda_values = logspace(-7, 2, 10); % 50 valeurs pour une meilleure précision

```

```

erreur_reconstruction_values = zeros(size(lambda_values));

% Boucle pour trouver le lambda optimal
for i = 1:length(lambda_values)
    lambda = lambda_values(i);

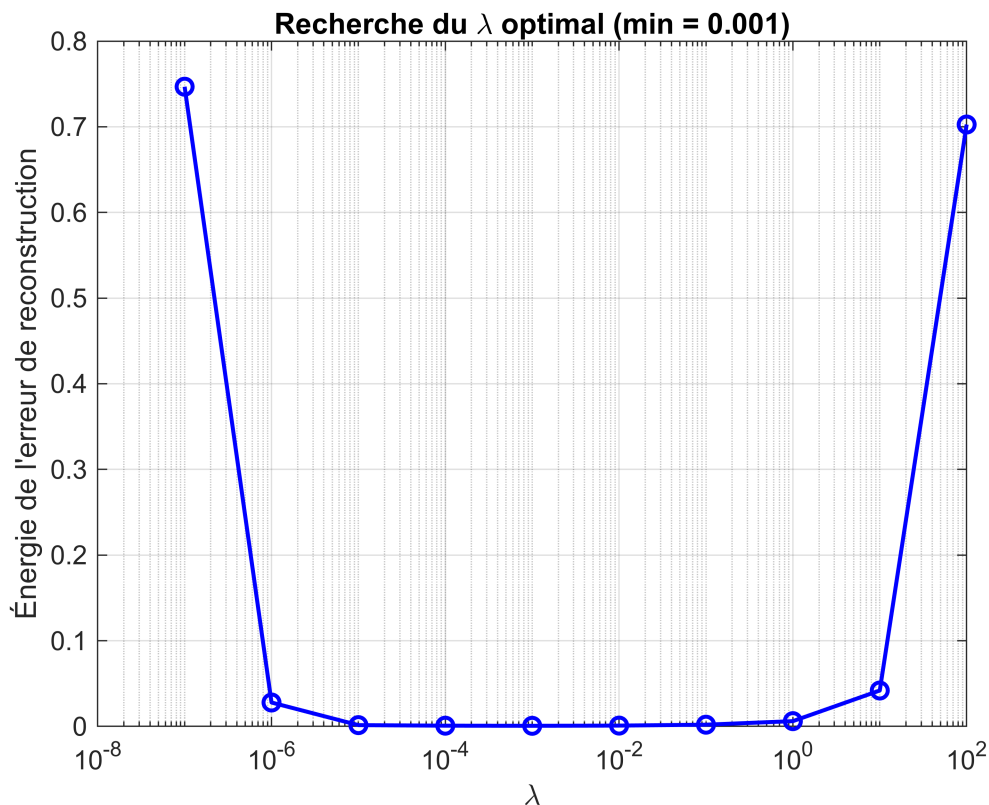
    % Résolution du problème régularisé
    x_rec_reg = (H' * H + lambda * (D1' * D1)) \ (H' * y');

    % Calcul de l'erreur de reconstruction
    erreur_reconstruction = x - x_rec_reg(1:length(x))';
    erreur_reconstruction_values(i) = sum(erreur_reconstruction .^ 2);
end

% Trouver le lambda qui minimise l'erreur
[~, idx_opt] = min(erreur_reconstruction_values);
lambda_optimal = lambda_values(idx_opt);

% Affichage de l'évolution de l'erreur en fonction de lambda
figure;
semilogx(lambda_values, erreur_reconstruction_values, 'bo-', 'LineWidth', 1.5);
xlabel('\lambda');
ylabel('Énergie de l'erreur de reconstruction');
title(['Recherche du \lambda optimal (min = ' num2str(lambda_optimal) ')']);
grid on;

```



Zone des fonctions fait et utilisé dans le TP

```
function x = SignTrigAS(L,tx)
% Génération du signal triangulaire
L2 = L/2; % Moitié de la durée d'observation
x1 = (tx(tx <= L2) / L2);           % Première partie croissante de 0 à L2
x2 = (- (tx(tx > L2) - L) / L2);    % Deuxième partie décroissante DE L2 à L
% Concaténation des deux parties
x = [x1, x2];
end
```

FIN ET EXPORTATION

```
% publish ("dmLucianoSoma.mlx", format = " pdf ");
```

J'ai eu : Error using publish

Unable to publish "dmLucianoSoma.mlx". Using the publish function to publish a MATLAB code file (.m) from a live script or function is not supported.

J'ai donc utilisé Export

```
export ("dmLucianoSoma.mlx", format = " pdf ");
```

```
Error using export (line 74)
Unable to find file.
```